BAN404 R code explanation Task 1

from probable to possible code explanation task 1

Self-study preparation file

2026-01-01

Table of contents

1 TASK 1 CHEAT SHEET: "Explain the Code" Dictionary

How to use this section in the exam: 1. Look at the code the professor gives you. 2. Scan the code for "The Tell" (the specific keyword or math symbol that gives away the secret). 3. Find the matching code in the table. 4. Go to that section and read the theory/explanation.

1.1 QUICK LOOKUP TABLE (TELL → METHOD)

If you see this in code…	It is almost certainly…	Section
sample(…, replace=TRUE)	Bootstrap / Bagging	1 / 14
for(i in 1:n) + X[-i,]	LOOCV	3
for(i in 1:K) + folds == i	K-Fold CV	2
sum((y - X%*%b)^2) + lasum(b^2)	Ridge	4
sum((y - X%*%b)^2) + lasum(abs(b))	Lasso	6
solve(t(X)%*%X)	OLS closed-form	7
abs(x-x0) + order(d)[1:K]	KNN	8
y - f2 / y - f1 in loop	Backfitting (GAM)	10
table(y[o])	KNN classification	9
exp(eta)/(1+exp(eta))	Logistic sigmoid	17
log(p/(1-p))	Log-odds	18
for(s in candidates) + RSS/Gini	Tree split	12
mincut / pruning loop	Tree tuning	13
mtry in randomForest	Random forest	15
r <- y - fhat in loop	Boosting	16
nested loops over cost/gamma	SVM tuning	21
prcomp() + pca$x	PCR	23
ROC / AUC code	ROC/AUC	24
cluster assignment + centroid update	K-means	25

1.2 GROUP 1: The Resampling Loops (Shuffling the Data)

1.2.1 1. The Bootstrap

Math / Theory: We want the sampling distribution and standard error of a statistic. We simulate B datasets of size n **with replacement** and compute the statistic each time. This is used when analytical formulas are too hard.

Why it matters (model improvement): The bootstrap gives *uncertainty estimates* around parameters or performance, which is essential for inference and for comparing models honestly.

The Raw Code you might see:

```r
f_boot <- function(data, B=1000) {                # Function takes data and number of simulations
  n <- nrow(data)                                 # Number of observations
  res <- numeric(B)                               # Container for the B simulated statistics
  for(i in 1:B) {                                 # Loop over simulations
    idx <- sample(1:n, size=n, replace=TRUE)      # THE TELL: sample WITH replacement (bootstrap)
    fake_data <- data[idx, ]                      # Create a bootstrap sample
    model <- lm(y ~ x, data=fake_data)            # Fit model on bootstrap sample
    res[i] <- coef(model)[2]                      # Save the statistic (e.g., beta1)
  }
  return(sd(res))                                 # Standard error of the statistic
}
```

The Tell: `sample(..., replace = TRUE)`.

Plug-and-Play Answer: > "This R-function implements the **Bootstrap** method. It estimates the standard error of the [INSERT METRIC]. It runs a `for` loop `B` times, and in each iteration it draws a random sample of size n with replacement. It then computes and saves the statistic. Finally, it returns the standard deviation of these B estimates."

Typical follow-up questions (and how to solve): - *"Compute a 95% CI using bootstrap (normal or percentile)."*

```r
# Suppose (assume) res contains bootstrap statistics
se_boot <- sd(res)

# Normal approximation CI
ci_normal <- mean(res) + c(-1, 1) * 1.96 * se_boot

# Percentile CI
ci_percentile <- quantile(res, c(0.025, 0.975))
```

Theory Answer:
The **normal approximation CI** assumes the bootstrap distribution is approximately normal, so we use mean $\pm$ 1.96 $\times$ SE. This is fast and standard but may be inaccurate if the bootstrap distribution is skewed.

The **percentile CI** uses the empirical quantiles of the bootstrap distribution directly, requiring no normality assumption. This is generally more robust when the distribution is asymmetric.

1.2.2 2. K-Fold Cross Validation

Typical Exam Question (Expanded)

> "Explain what the following function is doing. Which method is it an example of? Why is it used? If K changes, what trade-offs occur?"

Goal of the Task

Estimate how well a model will **generalize to new data**, and choose hyperparameters (like polynomial degree, K in KNN, or in ridge).

Assumptions

- Observations are i.i.d. (independent and identically distributed).
- The dataset is large enough to split into K folds without losing too much training signal.
- Random fold assignment is representative of future data.

Why K-Fold CV (and not LOOCV or train/test)?

- **Better bias-variance balance** than LOOCV.
- More stable than one random train/test split.
- Less computationally heavy than LOOCV.

Math / Theory: Estimate Test MSE by splitting data into K folds, training on $K-1$, testing on 1, and averaging errors.

Why it matters (model improvement): CV is the **standard tool for model selection**. It estimates generalization error so we can choose models or hyperparameters that *actually* predict well on new data.

```r
f_kfold <- function(X, y, K=5) {                # X predictors, y response, K folds
  n <- length(y)                                # Number of observations
  folds <- sample(rep(1:K, length=n))           # THE TELL: assign each row to a fold
  cv_errors <- numeric(K)                       # Container for fold errors

  for(i in 1:K) {                               # Loop through each fold
    train_X <- X[folds != i, ]                  # Training data (all but fold i)
    train_y <- y[folds != i]
    test_X  <- X[folds == i, ]                  # Test data (fold i)
    test_y  <- y[folds == i]

    b <- solve(t(train_X)%*%train_X) %*% t(train_X)%*%train_y  # Fit OLS
    preds <- test_X %*% b                       # Predict on test fold
```

```
    cv_errors[i] <- mean((test_y - preds)^2)    # MSE for this fold
  }
  return(mean(cv_errors))                       # Average CV error
}
```

The Tell: `for(i in 1:K)` with `folds != i` vs `folds == i`.

Plug-and-Play Answer: This function implements **K-fold cross-validation**, a method for estimating test error. The dataset is split into K roughly equal subsets called folds. The model is trained on K-1 folds and evaluated on the held-out fold. This repeats K times so every data point is tested exactly once.

The method produces an average test error that is more reliable than a single train/test split. Compared to LOOCV, K-fold has slightly more bias but **much lower variance**, making it a better choice in practice.

Typical follow-up questions (and how to solve): - *"Use CV to choose the best K or ."*

```
Kgrid <- 2:15                                   # Try candidate values of K (2,3,...,15)
mseK <- sapply(Kgrid, function(K)               # For each K, compute CV error
  f_kfold(X, y, K=K)                            # Run K-fold CV with that K
)
Kopt <- Kgrid[which.min(mseK)]                  # Pick the K giving smallest CV error
```

Theory answer: CV selects the hyperparameter that minimizes estimated test error, balancing bias/variance. We evaluate each candidate K using CV and select the one with the **lowest estimated test error**.

1.2.3 3. Leave-One-Out Cross-Validation (LOOCV)

Math / Theory: Special case of K-fold with $K = n$. Train on $n - 1$, test on 1, repeat for all rows.

Why it matters (model improvement): LOOCV gives a nearly unbiased estimate of test error for small datasets (at the cost of high variance). It is useful when data are scarce.

```
loo <- function(X, y) {                         # X predictors, y response
  n <- nrow(X)                                  # Number of observations
  ydpred <- matrix(0, n, 1)                     # Store 1 prediction per loop

  for(i in 1:n) {                               # THE TELL: loop over every row
    train_X <- X[-i, ]                          # Train on all but row i
    train_y <- y[-i]
    b1 <- g(X = train_X, y = train_y)           # Fit model on n-1 observations
    ydpred[i] <- X[i, ] %*% b1                  # Predict for left-out row i
  }

  testMSE <- mean((y - ydpred)^2)               # LOOCV MSE
  return(testMSE)
}
```

The Tell: `for(i in 1:n)` and `X[-i, ]`.

Plug-and-Play Answer: > "This R-function implements **LOOCV**. It trains the model on all observations except one, predicts the left-out observation, and repeats for every row. The returned value is the average Mean Squared Error (MSE) across all n test cases."

Typical follow-up questions (and how to solve): - *"Use LOOCV to pick in ridge/lasso."*

```
lavec <- seq(0, 5000, length.out=20)            # Candidate lambda values
mses <- sapply(lavec, function(la)              # For each lambda, compute LOOCV error
  loo(X, y)                                     # Run LOOCV (or loo(la,X,y) if la used)
)
la_opt <- lavec[which.min(mses)]                # Pick lambda with smallest LOOCV error
```

Theory answer: LOOCV approximates test error; the that minimizes LOOCV is preferred for generalization. We compute LOOCV error for each and choose the one that minimizes estimated test error.

`loo(la, X, y)` **if is defined in the function (tuning param)**
`loo(X, y)` **if is NOT in the function**

1.3 GROUP 2: Loss Functions & Optimization (Phase 3 Methods)

1.3.1

1.3.2 4. Ridge Regression Engine (Exam 2025 style)

Typical Exam Question

> "Explain what the following R functions are doing. Which method do they implement? Why is the data demeaned? Compare results to OLS when = 0 and when is large."

Math / Theory: Ridge minimizes $RSS + \lambda \sum \beta_j^2$. All variables must be centered so the penalty is fair. This prevents large-scale predictors from dominating the penalty.

The goal is to estimate regression coefficients **with regularization**, so the model is more stable and less sensitive to multicollinearity or noisy predictors.

Why it matters (model improvement): Ridge reduces **variance** by shrinking coefficients. This improves prediction when predictors are correlated or when p is large.

```r
# PART A: Loss function
f <- function(b1, X, y, la) {                    # b1 = coefficients, la = lambda
  fval <- sum((y - X%*%b1)^2) + la * sum(b1^2)   # THE TELL: RSS + lambda*sum(beta^2)
  return(fval)
}

# PART B: Data prep + optimizer
g <- function(X, y, la) {
  q <- ncol(X)                                   # Number of predictors
  b0 <- mean(y)                                  # Intercept trick: mean of y
  yd <- y - b0                                   # Center y to remove intercept

  Xd <- X
  for(k in 1:q) {                                # Demeaning loop (critical!)
    Xd[,k] <- X[,k] - mean(X[,k])                # Center each predictor
  }

  b1start <- rep(0, q)                           # Start betas at 0
  opt <- nlminb(b1start, f, X=Xd, y=yd, la=la)   # Optimize loss function
  return(opt$par)                                # Return ridge coefficients
}
```

The Tell: `sum((y - X%*%b)^2) + la * sum(b^2)` and the demeaning loop.

Plug-and-Play Answer:

This code implements **ridge regression**, which minimizes the penalized objective
RSS+ j j2The RSS term forces the model to fit the data, while the penalty shrinks coefficients. This shrinkage reduces variance, especially when predictors are correlated or when the model is high-dimensional.

The function `g` centers y and each column of X because ridge penalties are applied to coefficients, and we want that penalty to treat all predictors fairly. If variables are not centered, the intercept and predictors with large scales would be penalized unfairly.

By optimizing the ridge objective, we obtain coefficients that are **more stable** than OLS. When = 0, ridge reduces to OLS. As increases, coefficients shrink toward zero, giving a smoother, less variable fit.

Typical follow-up questions (and how to solve): - *"Compare OLS vs ridge; what happens as increases?"*

```
b_ols <- lm(y ~ X)$coef[-1]
b_ridge0 <- g(X, y, la=0)
b_ridge1000 <- g(X, y, la=1000)
cbind(b_ols, b_ridge0, b_ridge1000)
```

Theory answer: =0 gives OLS. Larger shrinks coefficients and reduces variance but increases bias. This is the core bias-variance trade-off.

1.3.3 5. Ridge with Standardization (often required in practice)

Math / Theory: Standardization makes variables comparable by subtracting the mean and dividing by the standard deviation: $\tilde{x}_{ij} = (x_{ij} - \bar{x}_j)/s_j$.

Why it matters (model improvement): If predictors are on different scales (e.g., income vs. age), ridge/lasso penalties will unfairly shrink some variables more than others. Standardization fixes this.

```
Xstd <- X
for(k in 1:ncol(X)) {
  Xstd[,k] <- (X[,k] - mean(X[,k])) / sd(X[,k])  # Center + scale each predictor
}
```

The Tell: Division by `sd()` after subtracting the mean.

Plug-and-Play Answer: > "This code **standardizes** the predictors before applying a shrinkage method. It ensures the penalty treats all variables fairly regardless of units, which is crucial for Ridge/Lasso."

Typical follow-up questions (and how to solve): - *"Why not only center?"* — Centering aligns means; scaling aligns variances so penalties are comparable.

1.3.4 6. Lasso Regression Engine (Probable 2026)

Math / Theory: Lasso minimizes $RSS + \lambda \sum |\beta_j|$ and can shrink coefficients to exactly 0.

Why it matters (model improvement): Lasso performs **variable selection**, producing simpler models that can improve interpretability and prediction when many predictors are irrelevant.

```
f_lasso <- function(b1, X, y, la) {              # b1 = coefficients
  fval <- sum((y - X%*%b1)^2) + la * sum(abs(b1))# THE TELL: absolute value penalty
  return(fval)
}
```

The Tell: `abs(b1)` instead of `b1^2`.

Plug-and-Play Answer: > "This function computes the **Lasso** objective: RSS plus the L1 penalty $\lambda \sum |\beta|$. The absolute value penalty allows coefficients to become exactly zero, so Lasso performs variable selection."

Typical follow-up questions (and how to solve): - *"Which predictors are selected?"* — those with non-zero coefficients after tuning .

1.3.5 7. Ordinary Least Squares (OLS) Matrix Formula

Math / Theory: $\hat{\beta} = (X^T X)^{-1} X^T y$.

Why it matters (model improvement): OLS is the baseline estimator. It has lowest bias when the linear model is correct, but can have high variance with many correlated predictors.

```
f_ols <- function(X, y) {
  b <- solve(t(X) %*% X) %*% t(X) %*% y          # THE TELL: (X'X)^-1 X'y
  return(b)
}
```

The Tell: `solve()` with `t(X) %*% X`.

Plug-and-Play Answer: > "This code computes **OLS regression coefficients** using the closed-form matrix solution $(X^T X)^{-1} X^T y$. The `solve()` function inverts the matrix."

1.4 GROUP 3: The 2024 & 2025 Actual Exam Functions

1.4.1 8. K-Nearest Neighbors (KNN) Local Regression (Exam 2024 style)

Math / Theory: Find the K closest points to x_0 and fit a local model on only those neighbors.

Why it matters (model improvement): KNN adapts to non-linear patterns; it can reduce bias when the true relationship is complex.

```
f_knn <- function(x0, x, y, K=3) {
  d <- abs(x - x0)                                # THE TELL: distance from x0 to all x
  o <- order(d)[1:K]                              # Indices of K closest points

  xl <- x[o]                                      # K closest x values
  yl <- y[o]                                      # K closest y values
  xydata <- data.frame(xl=xl, yl=yl)

  reg <- lm(yl ~ xl, data=xydata)                 # Fit local regression on neighbors
  ypred <- predict(reg, newdata=data.frame(xl=x0, yl=1))
  return(ypred)
}
```

The Tell: `abs(x - x0)` and `order(d)[1:K]`.

Plug-and-Play Answer: > "This function implements **KNN local regression**. It calculates distances from `x0`, selects the K nearest observations, fits a local regression on those K points, and predicts the value for `x0`."

Typical follow-up questions (and how to solve): - *"Choose K using LOOCV."*

```
Kgrid <- 1:15
mseK <- sapply(Kgrid, function(K) {
  preds <- sapply(1:n, function(i) f_knn(x[i], x[-i], y[-i], K))
  mean((y - preds)^2)
})
Kopt <- Kgrid[which.min(mseK)]
```

Theory answer: Small K → low bias/high variance; large K → smoother (higher bias/lower variance).

1.4.2 9. KNN Classification (Majority Vote)

Math / Theory: Estimate class probabilities by majority vote: $\hat{P}(Y = j|X = x_0) = \frac{1}{K} \sum_{i \in N_0} 1(y_i = j)$.

Why it matters (model improvement): Provides a flexible, non-parametric classifier without strong distributional assumptions.

```
knn_class <- function(x0, X, y, K=5) {
  d <- rowSums((X - matrix(x0, nrow(X), ncol(X), byrow=TRUE))^2)  # Distances
  o <- order(d)[1:K]                                              # K nearest
  pred <- names(sort(table(y[o]), decreasing=TRUE))[1]            # Majority vote
  return(pred)
}
```

The Tell: `table(y[o])` or majority vote on neighbors.

Plug-and-Play Answer: > "This function implements **KNN classification**. It finds the K nearest neighbors to `x0`, counts their class labels, and returns the majority class."

1.5 GROUP 4: Advanced Non-Linear Mechanics (Highly Probable Targets)

1.5.1 10. Backfitting Algorithm (GAMs)

Math / Theory: Iteratively fit each smooth term while holding others fixed: $y = f_1(x_1) + f_2(x_2)$.

Why it matters (model improvement): GAMs capture non-linear relationships while staying interpretable. Backfitting makes this possible by fitting one smooth at a time.

```
f_backfit <- function(x1, x2, y) {
  f1 <- rep(0, length(y))                    # Initialize f1
  f2 <- rep(0, length(y))                    # Initialize f2

  for(i in 1:10) {                           # Iterate to convergence
    f1 <- smooth.spline(x1, y - f2)$y        # Fit f1 on residuals (y - f2)
    f2 <- smooth.spline(x2, y - f1)$y        # Fit f2 on residuals (y - f1)
  }
  return(f1 + f2)                            # Final additive fit
}
```

The Tell: `y - f2` and `y - f1` inside a smooth.

Plug-and-Play Answer: > "This code implements **backfitting** for a GAM. It alternates between fitting a smooth function for x_1 on the residuals after removing f_2, then fitting f_2 on residuals without f_1. This continues until both functions converge."

Typical follow-up questions (and how to solve): - *"Compare GAM vs linear model."* — fit both and compare MSE / R^2.

1.5.2 11. Backfitting with Convergence Check (Optional variant)

Math / Theory: Same as backfitting, but stops when the change between iterations is small.

Why it matters (model improvement): Avoids unnecessary iterations and ensures stability of the fitted smooths.

```r
f_backfit_conv <- function(x1, x2, y, tol=1e-4) {
  f1 <- rep(0, length(y))
  f2 <- rep(0, length(y))
  diff <- Inf

  while(diff > tol) {                          # Stop when updates are small
    f1_new <- smooth.spline(x1, y - f2)$y
    f2_new <- smooth.spline(x2, y - f1_new)$y
    diff <- max(abs(f1_new - f1), abs(f2_new - f2))
    f1 <- f1_new; f2 <- f2_new
  }
  return(f1 + f2)
}
```

The Tell: `while(diff > tol)` and update comparison.

Plug-and-Play Answer: > "This is the **convergence version** of backfitting. It iterates until changes in the smooth functions are negligible, ensuring the additive model has stabilized."

1.6 GROUP 5: Tree-Based Mechanics (Probable + possible)

1.6.1 12. Tree Split Logic (Regression RSS or Classification Gini)

Math / Theory: Trees choose the split that minimizes impurity: RSS (regression) or Gini/Entropy (classification).

Why it matters (model improvement): The split that minimizes impurity creates the largest reduction in error at each step, making the tree fit the data better locally.

```r
best_split <- function(x, y) {
  candidates <- sort(unique(x))                # Candidate split points
  best_rss <- Inf                              # Start with very large RSS
  best_s <- NA

  for(s in candidates) {                       # Try each split point
    left  <- y[x < s]                          # Left region
    right <- y[x >= s]                         # Right region
    rss <- sum((left - mean(left))^2) +        # RSS left
           sum((right - mean(right))^2)        # RSS right

    if(rss < best_rss) {                       # Keep the best split
      best_rss <- rss
      best_s <- s
    }
  }
  return(best_s)
}
```

The Tell: Loop over split points and compute RSS or Gini for each.

Plug-and-Play Answer: > "This code implements **tree split selection**. It tests each candidate split point, computes impurity (here RSS), and chooses the split that minimizes impurity. This is the core logic of decision trees."

1.6.2 13. Tree Pruning / Complexity Tuning (mincut loop)

Math / Theory: Trees overfit if too deep. We control complexity by tuning parameters such as `mincut` or pruning size.

Why it matters (model improvement): Pruning reduces variance by cutting weak branches, which improves test performance.

```
mseres <- data.frame(mincut=2, testMSE=0)
for(k in 2:50) {
  tree_k <- tree(y ~ ., data=train, control=tree.control(nobs=nrow(train), mincut=k))
  pred_k <- predict(tree_k, newdata=valid)
  mseres[k-1,] <- cbind(k, mean((valid$y - pred_k)^2))
}
best_k <- mseres$mincut[which.min(mseres$testMSE)]
```

The Tell: Loop over `k` and track testMSE; choose best.

Plug-and-Play Answer: > "This loop **tunes tree complexity** by varying `mincut`. It chooses the value that minimizes validation MSE, which reduces overfitting and improves test performance."

1.6.3 14. Bagging Loop (Bootstrap Aggregation)

Math / Theory: Build many trees on bootstrap samples, then average predictions to reduce variance.

Why it matters (model improvement): Averaging many high-variance trees stabilizes predictions and reduces test error.

```
f_bagging <- function(X, y, B=100) {
  n <- nrow(X)
  preds <- matrix(0, nrow=n, ncol=B)             # Store B predictions

  for(i in 1:B) {
    idx <- sample(1:n, replace=TRUE)             # Bootstrap sample
    tree_mod <- tree(y ~ ., data=X[idx, ])       # Fit tree on bootstrap sample
    preds[, i] <- predict(tree_mod, newdata=X)   # Predict on full data
  }

  return(rowMeans(preds))                        # Average predictions across trees
}
```

The Tell: `sample(..., replace=TRUE)` inside a loop + model fit + `rowMeans()`.

Plug-and-Play Answer: > "This function implements **Bagging**. It creates many bootstrap samples, fits a decision tree to each, and averages the predictions. Averaging reduces variance and improves stability."

1.6.4 15. Random Forests (Feature Subsampling)

Math / Theory: Random forests add feature subsampling at each split (mtry) to decorrelate trees.

Why it matters (model improvement): Reducing correlation between trees improves the variance reduction from averaging.

```
rf <- randomForest(y ~ ., data=train, mtry=3, importance=TRUE)  # mtry < p
```

The Tell: `mtry` smaller than number of predictors.

Plug-and-Play Answer: > "This code fits a **Random Forest**. It is like bagging, but each split only considers a random subset of predictors (`mtry`). This decorrelates the trees and improves predictive performance."

1.6.5 16. Boosting Loop (Sequential Residual Fitting)

Math / Theory: Boosting builds trees sequentially on residuals: $\hat{f}(x) = \sum_{b=1}^{B} \nu \hat{f}_b(x)$.

Why it matters (model improvement): Boosting reduces **bias** by repeatedly correcting errors, often leading to strong predictive performance.

```
boost <- function(X, y, B=100, nu=0.1) {
  fhat <- rep(0, length(y))                    # Initial predictions

  for(b in 1:B) {                              # THE TELL: sequential loop
    r <- y - fhat                              # Residuals (errors)
    tree_b <- tree(r ~ ., data=X)              # Fit tree on residuals
    fhat <- fhat + nu * predict(tree_b, X)     # Update predictions with shrinkage
  }
  return(fhat)
}
```

The Tell: Residual update each loop (r <- y - fhat).

Plug-and-Play Answer: > "This function implements **boosting**. It starts with predictions 0, computes residuals, fits a weak learner to the residuals, and updates the prediction by adding a shrunken version of the new tree's predictions."

1.7 GROUP 6: Classification Mechanics (Highly Probable)

1.7.1 17. Logistic Regression Engine (Sigmoid / Log-Odds)

Math / Theory: $p(x) = \frac{e^{\eta}}{1+e^{\eta}}$ with $\eta = \beta_0 + \beta^T x$. Log-odds: $\log \frac{p}{1-p} = \eta$.

Why it matters (model improvement): Unlike linear regression, logistic regression produces valid probabilities between 0 and 1 and models classification directly.

```
logistic_pred <- function(X, b) {
  eta <- X %*% b                               # Linear predictor
  p <- exp(eta) / (1 + exp(eta))               # THE TELL: sigmoid function
  return(p)                                    # Predicted probability
}
```

The Tell: `exp(eta)/(1+exp(eta))`.

Plug-and-Play Answer: > "This code computes **logistic regression probabilities** using the sigmoid function. The linear predictor $\eta = X\beta$ is mapped to a probability by $p = \frac{e^{\eta}}{1+e^{\eta}}$."

1.7.2 18. Log-Odds Transformation (Interpretation)

Math / Theory: $\log \frac{p}{1-p} = \eta$.

Why it matters (model improvement): The log-odds scale is linear, which makes coefficients interpretable and suitable for estimation.

```
log_odds <- function(p) {
  return(log(p / (1 - p)))
}
```

The Tell: `log(p/(1-p))`.

Plug-and-Play Answer: > "This code transforms probabilities to **log-odds**, which is the linear scale used in logistic regression. This is why logistic regression coefficients have an additive interpretation on the log-odds scale."

1.7.3 19. Threshold & Confusion Matrix

Math / Theory: Classification by threshold: $\hat{y} = 1$ if $p > t$. Confusion matrix summarizes TP/TN/FP/FN.

Why it matters (model improvement): Changing the threshold trades off false positives vs false negatives. The confusion matrix quantifies performance.

```
prob <- predict(model, newdata=test, type="response")  # Predicted probabilities
pred <- prob > 0.5                                      # THE TELL: threshold rule
conf <- table(truth=test$y, predicted=pred)             # Confusion matrix
acc <- sum(diag(conf)) / sum(conf)                      # Accuracy
```

The Tell: `pred <- prob > 0.5` and `table(...)`.

Plug-and-Play Answer: > "This code converts predicted probabilities into class labels using a threshold (0.5). It then computes a confusion matrix and accuracy. This is how classification performance is evaluated."

1.7.4 20. LDA/QDA (Gaussian Classifier Assumptions)

Math / Theory: LDA assumes $X|Y = k \sim N(\mu_k, \Sigma)$ with shared covariance; QDA uses class-specific Σ_k.

Why it matters (model improvement): These methods can outperform logistic regression when Gaussian assumptions are approximately true.

```
mu1 <- colMeans(X[y==1, ])                    # Class 1 mean
mu0 <- colMeans(X[y==0, ])                    # Class 0 mean
Sigma <- cov(X)                               # Pooled covariance (LDA)
```

The Tell: class-wise means and covariance.

Plug-and-Play Answer: > "This code computes the parameters for **LDA/QDA**: class means and covariance. LDA uses a pooled covariance, while QDA would use separate covariances for each class."

1.8 GROUP 7: Possible (Less Likely) Code Chunks

1.8.1 21. SVM Tuning Grid (Cost / Gamma)

Math / Theory: Tune hyperparameters C and γ by CV to find best classification boundary.

Why it matters (model improvement): Choosing the right margin (C) and kernel flexibility (gamma) controls bias-variance tradeoff.

```
best <- Inf
for(C in c(0.1,1,10)) {
  for(g in c(0.5,1,2)) {
    fit <- svm(y ~ ., data=train, kernel="radial", cost=C, gamma=g)
    pred <- predict(fit, newdata=valid)
    err <- mean(pred != valid$y)
    if(err < best) {
      best <- err; bestC <- C; bestG <- g
    }
  }
}
```

The Tell: Nested loops over `cost` and `gamma` with error comparison.

Plug-and-Play Answer: > "This code performs a grid search to tune **SVM hyperparameters** (cost and gamma). It trains a model for each combination, evaluates validation error, and selects the parameters that perform best."

1.8.2 22. Polynomial / Step Function CV Loop

Math / Theory: Choose degree (or number of cuts) by comparing validation MSE.

Why it matters (model improvement): Prevents underfitting (too low degree) and overfitting (too high degree).

```
K <- 10
MSE <- numeric(K)
for(k in 1:K) {
  reg <- lm(y ~ poly(x, k), data=train)          # Polynomial degree k
  pred <- predict(reg, newdata=test)
  MSE[k] <- mean((test$y - pred)^2)
}
which.min(MSE)
```

The Tell: Loop over `k` and compute MSE for each.

Plug-and-Play Answer: > "This code uses validation to choose the **polynomial degree**. It fits models of increasing degree, computes test MSE for each, and selects the degree with the smallest MSE."

1.8.3 23. PCR Projection (Principal Components Regression)

Math / Theory: Project X onto top components $Z = XV$ and regress y on Z.

Why it matters (model improvement): Reduces dimensionality and multicollinearity, improving stability when predictors are highly correlated.

```r
pca <- prcomp(X, scale.=TRUE)                    # PCA on centered/scaled X
Z <- pca$x[,1:K]                                 # First K principal components
reg <- lm(y ~ Z)                                 # Regress y on components
```

The Tell: `prcomp()` + using `pca$x` as predictors.

Plug-and-Play Answer: > "This code performs **PCR**. It first computes principal components of X, then uses the first K components as predictors in a regression model. The goal is dimension reduction and reducing variance."

1.8.4 24. ROC / AUC Computation (Possible)

Math / Theory: ROC shows TPR vs FPR for thresholds; AUC summarizes ranking quality.

Why it matters (model improvement): Threshold-free evaluation for classification performance.

```r
library(pROC)
roc_obj <- roc(test$y, prob)
auc(roc_obj)
plot(roc_obj)
```

The Tell: `roc()` and `auc()`.

Plug-and-Play Answer: > "This code computes **ROC and AUC**, evaluating classification quality across all thresholds. AUC is the probability a random positive gets a higher score than a random negative."

1.8.5 25. K-Means Clustering (Possible)

Math / Theory: Alternates between assigning points to nearest centroid and updating centroids.

Why it matters (model improvement): Discovers structure without labels; useful for segmentation.

```r
# One manual iteration
clusters <- apply(X, 1, function(xi) which.min(colSums((t(centers) - xi)^2)))
centers <- sapply(1:K, function(k) colMeans(X[clusters==k, , drop=FALSE]))
```

The Tell: distance to centroids + centroid update.

Plug-and-Play Answer: > "This code performs one **k-means** iteration: assign points to closest centroid, then update centroids to the mean of assigned points."